

Modeling disaggregated GPT-OSS-120B decode on hybrid AMD MI455X + CS-4 system

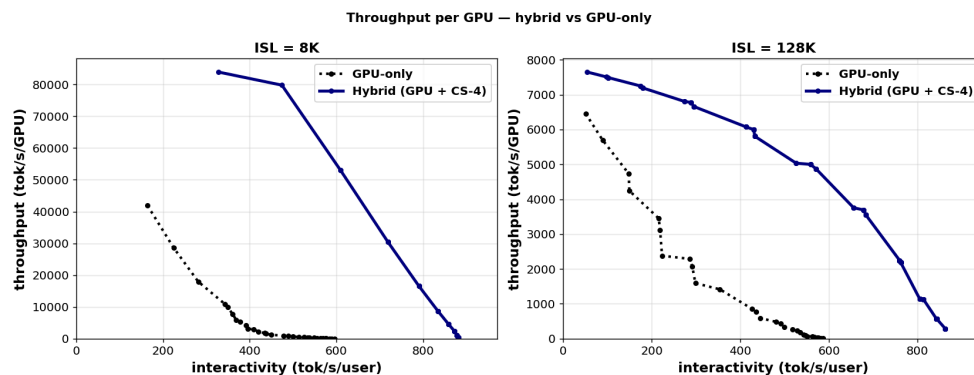
We model **decode** inference of GPT-OSS-120B on a disaggregated hybrid in which AMD **MI455X** GPUs execute attention and **two Cerebras CS-4 wafers** execute the remaining per-layer operations, coupled over a datacenter network. We characterize the throughput–interactivity and energy-efficiency trade-offs and identify the regimes in which disaggregation is advantageous.

Executive summary

Scope. We model the *decode* phase of GPT-OSS-120B inference on a disaggregated system in which MI455X GPUs execute attention and **two CS-4 wafers** execute the remaining per-layer operations (the projections, the mixture-of-experts router and experts), and compare it against a GPU-only baseline across context lengths.

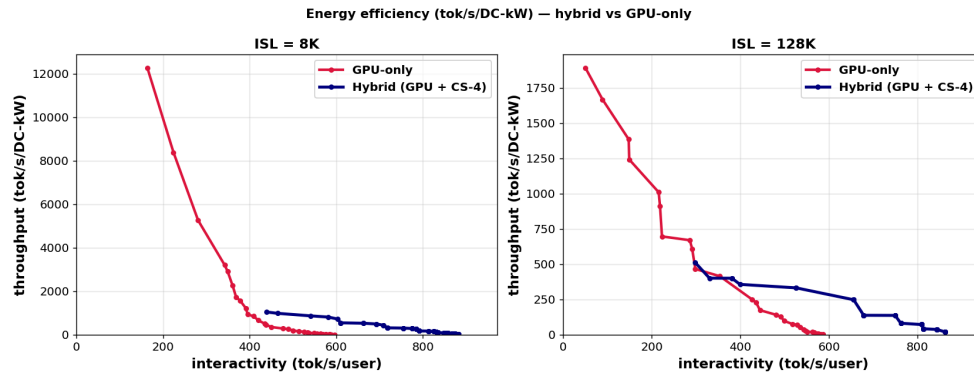
Methodology. The GPU model is fit to **measured** per-operation B200 decode timings and projected to MI455X by the memory-bandwidth ratio; it therefore retains real kernel-launch overheads and the sub-linear benefit of tensor-parallel sharding rather than idealized peak rates. The CS-4 model is analytical, with per-core timings **validated against Cerebras’s cycle-accurate simulator for CS-3 and scaled to CS-4**. We report two metrics: **throughput per GPU** (a hardware-cost proxy) and **throughput per datacenter-kilowatt** (energy efficiency). The figures below show 8K and 128K context; the full 8K–1M sweeps are in §5–§6.

1. Throughput per GPU — the hybrid is higher, with the largest margin in the interactive (low-latency) regime.



throughput per GPU

2. Energy per datacenter-watt — GPU-only is higher at peak; the hybrid is higher at high interactivity.



energy efficiency

Findings.

- **Throughput per GPU is higher for the hybrid** — by **2.0×** at 8K, and by larger margins across the interactive (small-batch) region, where the non-attention work dominates the GPU’s per-layer time and the wafer absorbs it. The peak-throughput margin narrows at long context as both systems become attention-bound ($\sim 1.14\times$ at 1M) but remains substantial through the interactive region (§5).
- **Energy efficiency exhibits a crossover.** At peak throughput (large batch) the GPU-only rack is more efficient — the hybrid carries a fixed 94 kW wafer cost — reaching $\sim 0.09\times$ of the GPU-only rack at 8K and $\sim 0.57\times$ at 1M. At high interactivity GPU-only efficiency degrades sharply (small batches spread over many active GPUs), and the hybrid becomes the more efficient configuration (§6).
- **Only the hybrid serves the highest interactivity.** GPU-only saturates near ~ 596 tok/s/user (~ 526 at 1M) — a per-user *latency* ceiling (1/TPOT), independent of how throughput is normalized — while the hybrid extends roughly $1.5\times$ further, to ~ 883 tok/s/user at 8K (and ~ 738 at 1M). Operating in that GPU-only-unreachable band needs **1–8 GPUs per CS-4** (the throughput-per-GPU-optimal choice — one GPU at short context, up to 8 at long context — trading per-GPU throughput for lower latency; §7).
- **Interactivity is sensitive to the GPU↔CS-4 link.** At the default $12\mu s$ the 8K hybrid frontier reaches ~ 883 tok/s/user — already $1.5\times$ the GPU-only ceiling of ~ 596 — and a faster link pushes it further to $\sim 1,090$ ($1.8\times$ GPU-only) at $6\mu s$ and $\sim 1,183$ ($2.0\times$ GPU-only) at $4\mu s$ (co-packaged) — the whole frontier shifts up-and-to-the-right (more throughput at a given high interactivity). A short link buys interactivity headroom, not peak throughput (§5).
- **Overall**, disaggregation is advantageous for **short-to-mid context, latency-sensitive** serving, where both per-GPU throughput and (in that regime) energy efficiency favor the hybrid; for long-context bulk throughput a GPU-only rack is more economical.

1. Disaggregated hybrid decode

“Decode” is the token-by-token generation phase of inference: the model emits one output token per step, and each step re-reads the whole model and the conversation so far. A transformer layer does two kinds of work, and this design puts each kind on the hardware that suits it best:

- **The GPU keeps attention** (the part where each token looks back at every earlier token through the “KV cache” — the saved keys and values). Attention is limited by how fast the GPU can *read memory* (it scans the whole KV cache every step) and by

how much memory it has (the cache grows with the conversation length). Fast, large GPU memory (HBM) is the right fit.

- **The Cerebras CS-4 wafer does everything else** — the input/output projections, the mixture-of-experts (MoE) router, and the expert feed-forward layers, plus the data shuffling those need. These are “weight-heavy” steps: they multiply by large fixed weight matrices that, on the wafer, sit permanently in on-chip memory and never have to be re-loaded. The CS-4 side is **two wafers and is never duplicated**.

Every layer the GPU hands its attention output to the wafer and gets fresh query/key/value vectors back, so the two trade data over a network link **twice per layer**. The model below times all three pieces — GPU attention, CS-4 ‘rest’, and the link between them — and asks how many GPUs should share the one wafer.

Notation & terms

This report keeps formulas and tables compact by using short symbols. They are all defined here once; the body text uses plain words wherever it can.

term	plain meaning
TPOT	time per output token (seconds) — the per-token decode latency. Lower is better.
tok/s/GPU	throughput divided by the number of GPUs — a per-accelerator efficiency (cost) measure.
tok/s/DC-kW	throughput per kilowatt of datacenter power — an energy-efficiency measure.
data-parallel (dp) groups	independent GPU groups each running a full model copy and sharing the workload; in formulas, a_g .
per-GPU microbatch	the small slice of the batch a GPU group processes at a time; in formulas, ck .
attention term	modeled GPU attention time; in formulas, a_{hbm} (full) and a_{hbm_slid} (sliding-window).
the ‘rest’	everything that isn’t attention (projections + router + experts); in formulas, r_{hbm} .
fixed overhead	the part of a step whose cost doesn’t shrink with faster memory (kernel launch, setup).
wafer / PE	the CS-4 is one silicon wafer of ~900k tiny cores (processing elements, PEs).

2. Architectures

The GPU side is modeled on **Helios MI455X** (see Appendix A for the projection method).

Helios MI455X	
HBM bandwidth	19.6 TB/s
HBM capacity	432 GB
FP4 peak	40 PF
FP8 peak	20.0 PF
BF16 peak	4 PF
scale-up link (bidirectional)	3.6 TB/s/GPU (UALink)
scale-up latency	0.7 μ s
rack power	246 kW
GPUs / rack	72

CS-4. One CS-4 system is a single wafer; a CS-4 **rack** holds **3** wafers (147 kW/rack). The expert weights of GPT-OSS-120B take about 57 GB in 4-bit form, which needs the on-chip memory of **2 wafers** — so the model runs on **two CS-4 wafers** (94 kW), housed in a rack that holds three. We never use more than these two wafers.

spec	per wafer (1 system)	two wafers (this model)	rack (3 wafers)
max power	47 kW	94 kW	147 kW (rack)
on-chip SRAM	44 GB	88 GB	132 GB
memory BW	41 PB/s	82 PB/s	123 PB/s
core-to-core fabric	418 Pb/s	—	—

*The hybrid is modeled on **two CS-4 wafers** (88 GB SRAM, 94 kW); the third wafer in the rack is unused by this workload.*

CS-4 per-PE / fabric parameters (the architectural inputs to the wafer model, Appendix B). The **CS-3** column is the simulator-validated baseline; **CS-4** is projected from it (see Appendix B):

parameter	CS-3 (sim-validated)	CS-4 (projected)
PEs / wafer	~901k (949 ²)	~901k (949 ²)
clock	1.1 GHz	1.1 GHz
FLOP / cycle / PE	16	32
peak FP16 (aggregate)	15.9 PFLOP/s	31.7 PFLOP/s
SRAM / PE	48 KB	48 KB
SRAM read / write	16 / 8 B/cy/PE	32 / 16 B/cy/PE
aggregate SRAM BW	21 PB/s	41 PB/s
PE↔PE BW	4 B/cy	8 B/cy
PE↔PE latency	10 cy (~9 ns)	20 cy (~18 ns)
K-tree all-reduce startup	7 cy	14 cy
off-wafer / inter-wafer I/O	1.2 Tbps (150 GB/s)	2.4 Tbps (300 GB/s)
inter-wafer link latency	—	3.0 μ s one-way (assumed)

CS-4 doubles CS-3 on every rate (FLOP/cy 16→32, SRAM r/w and PE↔PE BW ×2, I/O 1.2→2.4 Tbps); the comm startup also doubles (7→14 cy). Per-PE SRAM (48 KB) and clock (1.1 GHz) are unchanged.

The GPU↔CS-4 link. The GPU and wafer sit in different racks, connected through a multi-tier switch network (the kind used between racks in a datacenter). The round-trip cost adds up from:

- each switch a message passes through: 800 ns, and the path crosses 5 switches one way; plus 1 μ s at each of the 2 network cards it traverses → about 6.0 μ s one way.
- there are 2 transfers per layer (send attention output, get query/key/value back), so the link adds **12 μ s per layer** of unavoidable latency.

3. Performance Modeling

Per layer the hybrid splits the work across two accelerators: the AMD **MI455X** GPUs run **attention** (FMHA — bound by KV-cache bandwidth and capacity, a fit for GPU HBM), and one 2-wafer Cerebras **CS-4** runs **the rest** (the projections, the mixture-of-experts router and experts, and the on-wafer data movement — weight-resident work that lives in on-chip SRAM). Each accelerator is modeled on its own terms.

GPU model (measured). Built from measured NVIDIA B200 per-operation decode times and projected to MI455X by the memory-bandwidth ratio, so it carries real kernel-launch overhead and the sub-linear benefit of tensor-parallel sharding rather than idealized peak rates. It reproduces the measured data to ~4% (leave-one-out) and matches the external InferenceX serving benchmark within a few percent. Full bottleneck classes, fitted equations, and projection rules are in **Appendix A**.

CS-4 model (analytical). A wafer-scale GEMV + on-wafer-movement model (after WaferLLM) whose per-core timings were validated against Cerebras’s cycle-accurate simulator for CS-3 and scaled to CS-4. Its non-attention cost is nearly batch-independent because the expert weights stay resident on-chip. Details are in **Appendix B**.

The complete model derivations and all validation (leave-one-out, scaling checks, the InferenceX and overlap-adjusted comparisons) are in **Appendices A–C**. The body below (§4–§7) composes the two models into the hybrid pipeline and sweeps the throughput/efficiency Pareto.

4. Hybrid disaggregation & Pareto methodology

How the pipeline works. The work is streamed through a four-stage loop — GPU attention → link → CS-4 rest → link — in small slices (per-GPU microbatches). While the wafer works on one slice, the GPUs are already computing the next, so the link and the two stages overlap as in a pipeline. Several GPU groups can share the one wafer. In symbols — a = GPU attention time/layer, f = CS-4 rest time/layer, c = one-way link time; gb = global batch, a_g = number of data-parallel GPU groups, tp_g = GPUs per group (tensor-parallel width), ck = per-GPU microbatch, s = pipeline stages:

$$\begin{aligned}
 L_{\text{chunk}} &= a + 2c + a_g \cdot f && \text{(per-microbatch round-trip latency)} \\
 \text{block} &= \max(s \cdot a, a_g \cdot s \cdot f, L_{\text{chunk}}) && \text{(steady-state per-layer time)}
 \end{aligned}$$

```

throughput    = gb / (L·block)           # tok/s    (link hidden once
the pipe is full)
interactivity = 1 / (L·L_chunk)         # tok/s/user (per-token
latency, link exposed)

```

Two things to read off this: once the line is full, **throughput is set by the slowest stage** (the link disappears into the overlap), while **per-user latency is one full round trip** (the link is fully exposed there).

How we find the best configurations. We try **every** valid combination of total workload, tensor-parallel width, number of dp groups, and microbatch size — we do *not* hand-pick balanced ones — and keep the best trade-offs (the configurations that aren’t beaten on both throughput and interactivity at once). Well-balanced pipelines simply show up on that frontier by themselves.

On batch sizes — one workload, split among dp groups. We start from a single **global batch** of up to 256 sequences (the same ceiling the GPU-only baseline uses). To keep the GPUs and the wafer both busy, the GPU side is duplicated into several **data-parallel (dp) groups**, and the global batch is divided evenly among them; each dp group then streams its share through the pipeline in microbatches:

```

global_batch  gb = a_g · ck · s          (a_g = dp groups, ck = per-GPU
microbatch, s = pipeline stages)
per-group batch = gb / a_g = ck · s

```

A microbatch is never bigger than a dp group’s share, which is never bigger than the whole workload. More dp groups means a smaller share each, so at very long context (where attention is heavy) we use many dp groups with small microbatches to keep the single wafer fed.

Why the operating point matters: batch size hits attention and ‘the rest’ differently.

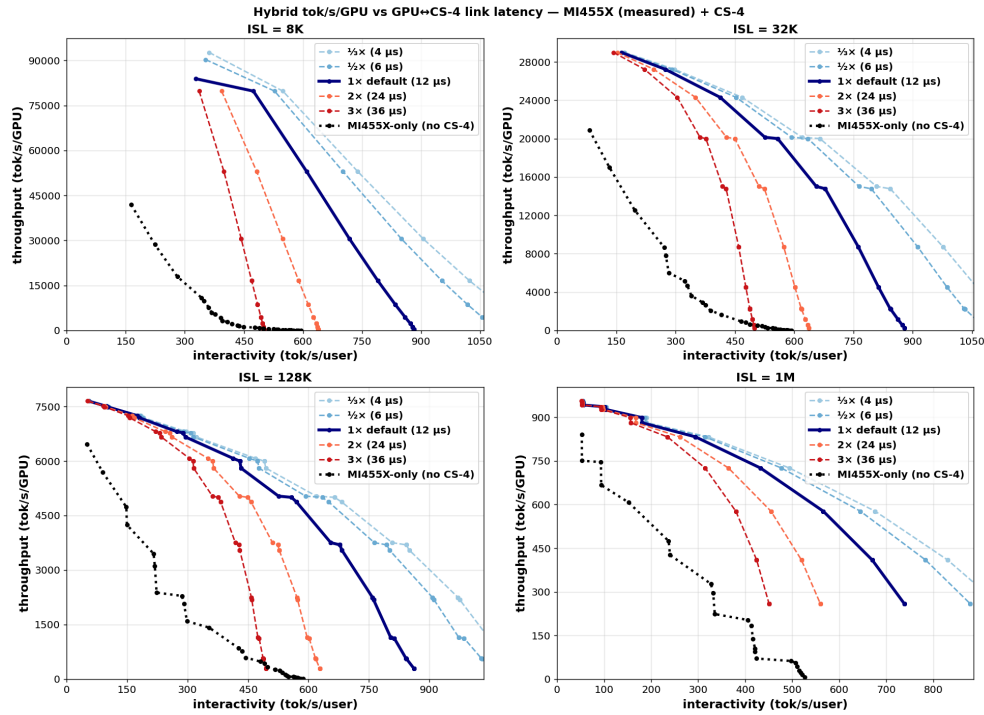
- **Attention grows with batch:** each sequence reads its own KV cache, so doubling the batch doubles the attention work. Per token it stays flat, but the total rises.
- **The rest barely grows with batch:** the projections and experts read their weights once and apply them to the whole batch, so per token the cost *falls* as batch rises, and the total is nearly flat (it only inches up as more experts get activated).

So **small batches are dominated by ‘the rest’ and large batches by attention**. At a 1M-token context, the rest is about **65%** of a layer at batch 4 but only ~6% at batch 256. Since high interactivity forces small batches, this is exactly where moving the rest onto the wafer helps most (§5).

The two systems we compare. The **hybrid** is GPUs (running attention) plus the one shared wafer (running the rest). The **GPU-only baseline** is the same measured model doing *all* the work on GPUs, with no wafer and no link. The wafer is never duplicated — adding capacity means adding dp groups around the one wafer, which is why long context (attention-heavy) wants many GPUs per wafer and short context wants few.

Two ways to score them. §5 reports **tokens/sec per GPU** — this doesn’t depend on how many dp groups we run, so it is a fair per-accelerator comparison (the wafer is not counted as a GPU). §6 reports **tokens/sec per datacenter-kilowatt**, which *does* charge for the wafer’s 94 kW of power.

5. Pareto — output tok/s/GPU



hybrid tok/s/GPU Pareto grid

Four contexts (8K, 32K, 128K, 1M), two panels per row. Each curve is the best hybrid trade-off at a different GPU↔CS-4 link latency — **solid navy** is the default (12 μs); the **dashed** curves vary it from 1/2× to 3× (a co-packaged link through a long-haul one). The **dotted black** line is GPU-only (no wafer, so link latency doesn't apply). A shorter link mainly extends the right-hand end (how fast a single user can be served); it barely changes the peak tokens/sec per GPU.

Peak tok/s/GPU — hybrid vs MI455X-only:

ISL	hybrid	MI455X-only	hybrid/GPU	winner
8K	83,921	41,896	2.00×	hybrid
32K	28,973	20,897	1.39×	hybrid
128K	7,649	6,459	1.18×	hybrid
1M	956	842	1.14×	hybrid

The result: the hybrid reaches fast-response operating points a GPU-only rack physically cannot. A GPU-only rack has a hard **interactivity ceiling** — its black curve ends at ~596 tokens/sec/user at 8K and ~526 at 1M — beyond which it would need fewer than one sequence per GPU and simply stops. The hybrid continues well past it, to ~883 (8K) and ~738 (1M): the wafer lifts the non-attention work off the GPU's critical path, so the GPUs stay productive at the tiny batches fast responses require. Wherever the two overlap, the hybrid also sits far higher in tok/s/GPU **at the same latency** — e.g. at 1M / 300 tokens/sec/user, 833 vs 327 tok/s/GPU (2.5×), i.e. several-fold fewer GPUs for the same response speed. This holds across **all four contexts**. The curves converge only at the **peak-throughput** end (large batch, low interactivity — at 1M both ~956/842 tok/s/GPU), where both are attention-bound; the hybrid's value is specifically in the interactive regime.

The mechanism: to deliver higher interactivity the GPU-only system must shrink the batch, and at small batch two effects degrade its per-GPU throughput:

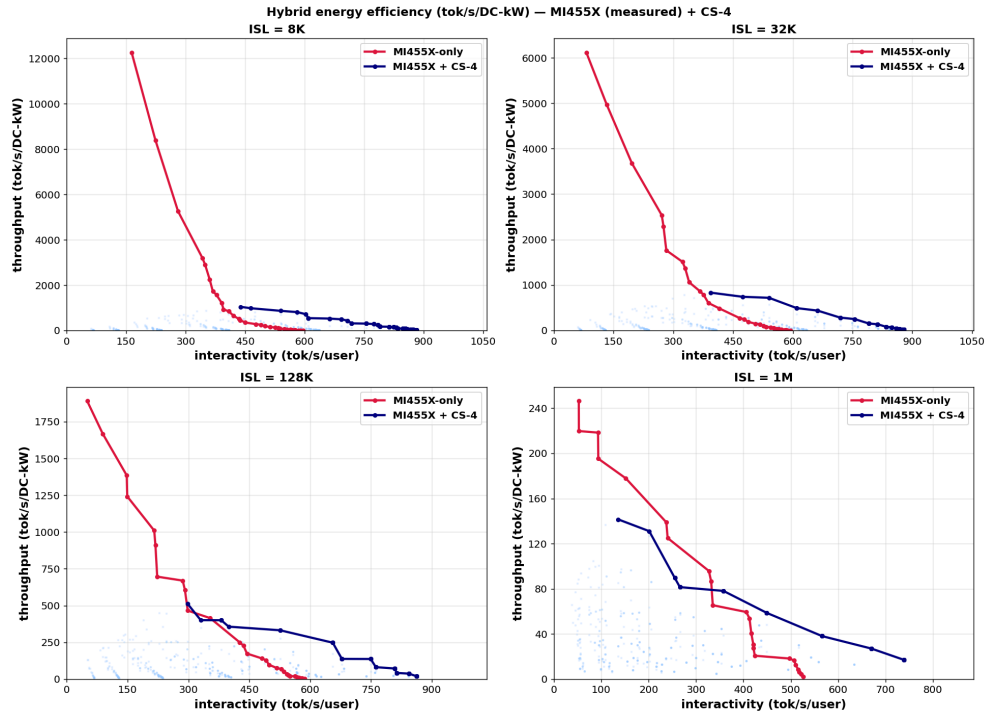
- **The non-attention work ceases to be negligible.** As shown in §4, it is nearly independent of batch, and most of it is overhead that a faster GPU does not accelerate. At large batch it is a small fraction of the layer (~94% is attention at batch 256), but at the small batches needed for low latency it *dominates*: at batch 4 / 8K tokens a GPU spends about **7 μ s on attention vs 44 μ s on the rest** (38 fixed overhead + 5 weight-reading + 1 all-reduce). The hybrid moves this work to the wafer, overlapped with the next microbatch's attention, so its GPUs spend essentially all their time on attention.
- **Tensor-parallel sharding becomes inefficient.** To increase interactivity the GPU-only system distributes each layer over more GPUs, but the fixed overhead does not shrink when split, so more GPUs share the same small batch and per-GPU throughput falls sharply. The hybrid reaches the same interactivity with **fewer** GPUs — e.g. at 300 tokens/sec/user it delivers 833 tokens/sec/GPU versus 327 for GPU-only.

This also explains the behavior at **1M tokens**, where attention is often described as dominant: it dominates only at *large* batch — attention grows with batch while the rest stays nearly fixed, so at 1M it is ~94% of a layer at batch 256 but only ~35% at batch 4. High interactivity requires small batches, where the rest is again the majority; the two curves therefore coincide at the large-batch peak (left) and diverge across the interactive, small-batch region (right).

In summary, the wafer absorbs the batch-independent work, so the GPU does not trade attention throughput for it. This metric does not charge for the wafer and therefore favors few GPUs per wafer; the energy view (§6) and the provisioning analysis (§7) determine the operating balance.

A faster link widens the win (the dashed curves). The GPU↔CS-4 link sits on the per-token critical path, so shortening it extends how far right the hybrid reaches — enlarging the region no GPU-only rack can match. At 8K the hybrid's reachable interactivity is ~883 tokens/sec/user at the default 12 μ s link — 1.5× the GPU-only ceiling of ~596 — rising to ~1,090 (1.8×) at 6 μ s and ~1,183 (2.0×) at 4 μ s (co-packaged); at 1M ~738 (1.4× the ~526 ceiling) rising to ~938 (1.8×) at 4 μ s. The effect is largest at short context (the link is a bigger share of a small round-trip). Crucially, even a 3× (36 μ s) link still clears GPU-only across the interactive range — so the win is robust to the interconnect, and a co-packaged link only makes it bigger.

6. Pareto — output tok/s/DC-kW



hybrid tok/s/DC-kW Pareto grid

Four contexts (8K, 32K, 128K, 1M), two panels per row: hybrid (navy) vs GPU-only (crimson), with the faint dots showing all hybrid configurations behind the frontier.

How power is computed. Each config's DC power is the GPUs plus the two CS-4 wafers:

```
hybrid    pw = a_g · tp_g · P_MI455X + P_CS4_2WAF
GPU-only  pw = tp_g · P_MI455X
P_MI455X  = 246 kW rack / 72 GPUs = 3.42 kW/GPU    (rack-amortized:
compute + cooling + scale-up network share)
P_CS4_2WAF = 2 × 47 kW = 94 kW    (two CS-4 wafers; datasheet max
system power)
tok/s/DC-kW = throughput / pw
```

The hybrid carries a fixed **94 kW** wafer power cost on top of its GPUs; the GPU-only baseline has no such term.

Peak tok/s/DC-kW — hybrid vs MI455X-only:

ISL	hybrid	MI455X-only	hybrid/GPU	winner
8K	1,043	12,262	0.09×	GPU
32K	832	6,116	0.14×	GPU
128K	513	1,890	0.27×	GPU
1M	141	246	0.57×	GPU

Analysis. At the **peak** (large-batch, low-interactivity; left of each panel) the **GPU-only rack is more energy-efficient** — the wafer’s fixed 94 kW is amortized over at most 72 GPUs — and this peak gap narrows with context (the wafer’s work is small relative to attention at 1M) without closing. **The two curves cross, however:** at the **high-interactivity** end (right) GPU-only efficiency degrades toward zero tokens-per-watt — a small batch is spread over many active GPUs — while the hybrid continues to produce useful work, so beyond the crossover the **hybrid is the more energy-efficient** configuration (the hybrid curve lies above GPU-only on the right of each panel). In summary, disaggregation improves **per-GPU throughput** across the range (§5) and **energy efficiency in the interactive regime**, but not at peak throughput, where the fixed wafer power dominates.

7. GPU : CS-4 ratios for the interactivity GPU-only cannot reach

GPU-only decode has an **interactivity ceiling**: beyond a certain tokens/sec/user it cannot operate at all. To go faster it must shrink the batch and shard each layer over more GPUs, but the non-attention work and the all-reduce impose a per-token latency floor that shrinking the batch cannot beat — past the ceiling a configuration would need fewer than one sequence per GPU. The hybrid extends past this ceiling because the wafer removes the non-attention work from the GPU’s critical path, leaving the GPU on attention alone. This section reports, per context, the interactivity band **only the hybrid can serve** and the GPU:CS-4 provisioning along it.

Interactivity reachable only by the hybrid:

context	GPU-only ceiling (tok/s/user)	hybrid ceiling (tok/s/user)	extra reach	GPs per CS-4 across the band
8K	596	883	1.48×	1 → 8
32K	594	879	1.48×	1 → 8
128K	586	863	1.47×	2 → 8
1M	526	738	1.40×	8 → 8

Reading the table. Entering the GPU-only-impossible region starts at a **single GPU** per two CS-4 wafers at short context (the wafer does almost the whole layer); climbing toward the ceiling adds tensor-parallel width (1×2, 1×4, 1×8), trading per-GPU throughput for lower latency. At 1M the band is served entirely at **8 GPUs per CS-4** (1×8): long-context attention already needs the full 8-way split, and the wafer extends the reachable latency beyond what those 8 GPUs could achieve alone.

Appendix A. GPU performance model (measured)

We do **not** model the GPU from first principles. Instead we start from **real measured timings** on an NVIDIA B200 — the per-operation decode times for 58 settings (tensor-parallel width 2/4/8, batch 1/4/16/64, prompt lengths 8K–256K), captured across all 36 layers — and then **project** those timings to the GPU we care about (MI455X). Because

the numbers are measured, they already include real-world costs that idealized models miss: per-operation launch overhead, and the fact that splitting work across more GPUs helps *less* than perfectly.

To project from one GPU to another, we sort each layer’s operations into a few **categories** that scale differently. The only thing that changes between GPUs is memory speed, so categories whose cost is set by memory bandwidth get faster by the **memory-speed ratio**, while fixed-overhead categories do not:

category (symbol)	what it is	how its cost grows	faster on a faster GPU?
attention, full (a_hbm)	reading the KV cache	with batch × prompt length	yes — scales with memory speed
attention, sliding (a_hbm_sld)	attention over a fixed 128-token window	flat in prompt length	yes
attention fixed cost (a_floor)	position encoding + cache write	fixed per step	no
rest — fixed overhead (r_hbm overhead)	launch/setup of projections, router, experts	fixed	no
rest — weight reading (r_hbm bandwidth)	reading the big weight matrices	with weights, shared across GPUs	yes
all-reduce (comm)	the cross-GPU sum tensor parallelism needs	with batch and GPU count	re-timed for the target link

We fit a simple formula to each category (least-squares on the measured grid, in microseconds per layer):

```

a_hbm      = 9.58 + (1.325e-04*B + 9.893e-05)*ISL/min(TP,8)  # full-
causal: grows with ISL
a_hbm_sld  = 6.34 + (-3.746e-03)*B                          #
sliding: flat in ISL (window=128)
r_hbm      = 37.5 (overhead floor, fixed) + (15.4 +
0.94*experts_active(B))/TP^0.40 (bandwidth, xHBM-ratio)
a_floor    = 3.95 + 0.0123*B (RoPE+KV; the coarse 6-bucket profile
folds the launch/quant/
route/finalize kernels into the r_hbm overhead, so there is
no separate r_floor term)
comm(μs/layer, NVLink) TP2=13.3 TP4=14.6 TP8=18.3 f_ar(comm
hidden by overlap) TP2=0.39 TP4=0.47 TP8=0.86

```

GPT-OSS-120B alternates two attention styles every other layer (18 full-history layers + 18 that only look back a fixed 128 tokens), so a layer’s attention time is the average of the two. The full per-layer time is then **attention (memory-scaled) + attention fixed cost + rest**, where the rest is its fixed overhead (unchanged across GPUs) + its weight-reading part (memory-scaled) + the all-reduce.

Why the ‘rest’ only partly speeds up on a faster GPU. The non-attention work has two parts that behave oppositely with batch size. At small batch it is dominated by **fixed overhead** — launching and setting up the operations — which a faster memory does *not* help. At large batch it is dominated by **reading the weight matrices**, which faster memory *does* speed up. So we keep them separate: the overhead stays put, and only the weight-reading part gets the target GPU’s memory-speed boost (here up to **2.45×**). The result: small batches barely speed up, large batches get nearly the full 2.45×. (One fitted detail: splitting the weight read across more GPUs helps less than proportionally — captured by a fitted sharding exponent of **0.40** rather than a perfect 1.0.)

The all-reduce partly overlaps with compute. The cross-GPU sum doesn’t sit entirely on the critical path — some of it runs while the GPU is still computing, so simply adding up operation times over-counts it. The measured profile reports how much overlaps, and we hide that fraction. The fraction grows with the number of GPUs sharing the layer (about 39% hidden at 2-way, rising to ~86% at 8-way). This adjustment is on by default and is what brings the model in line with the measured overlap-adjusted timings (Appendix C, item (f)); every projected number below uses it.

Appendix B. CS-4 performance model

The two CS-4 wafers run the **non-attention** part of every layer — the input/output projections, the mixture-of-experts router, and the expert feed-forward layers. The expert weights live permanently in the wafer’s 88 GB of on-chip memory, so unlike a GPU the wafer never re-loads them from slower memory each step. This side is modeled **analytically** (not from measured GPT-OSS traces); per layer it adds up three costs:

- **Computing the matrix multiplies** — done across the wafer’s ~900k cores reading weights from on-chip memory at 41 PB/s. Only the experts a token actually routes to are active: about 4 at batch 1, ~111 at batch 64, ~128 at batch 256.
- **Shuffling tokens to their experts** — the mixture-of-experts step has to gather each token to the right expert and broadcast inputs across the wafer; this data movement is counted explicitly.
- **Crossing between the two wafers** — roughly half the routed token copies have to hop from one wafer to the other.

The total depends only on the batch size (there are just two CS-4 wafers, never duplicated):

batch B	total μ s/layer	compute + on-wafer move	inter-wafer EP
8	13.4	7.1	6.3
64	20.4	11.9	8.5
256	42.2	26.3	15.8

Because the weights stay on-chip, this non-attention cost barely grows with batch size — the opposite of the GPU, whose non-attention work is dominated by fixed overhead at small batch (Appendix A). The link between the GPU and the wafer adds **12 μ s per layer** (§2), paid twice per layer as the two trade data.

Where the CS-4 model comes from, and how it’s validated. The wafer model follows the **WaferLLM** paper ([arXiv:2502.04563](https://arxiv.org/abs/2502.04563)) — both the way a matrix multiply is spread across the wafer’s cores and the way tokens are shuffled between them. The underlying per-core timings (how fast each core computes, reads its local memory, and talks to its

neighbors) were **checked against Cerebras’s cycle-accurate simulator for the CS-3** generation, then **scaled up to CS-4** using the published generation-over-generation improvements in the §2 spec table (compute and memory rates roughly doubled). So CS-3 is the validated anchor and **CS-4 is a projection from it** — mirroring how the GPU side measures a B200 and projects to MI455X (Appendix A). Only the GPU side is fit to actual GPT-OSS-120B traces; the wafer side is this simulator-anchored analytical model.

Appendix C. Validation

The GPU model is fit to measured data, so the question is how well it reproduces that data and known anchors.

(a) Does the fit generalize? (leave-one-out). Drop each of the 58 measured settings, re-fit without it, then predict it: average error **4.1%** (median 3.5%, worst 13.5%). Broken out by batch size:

batch	layer-time error	attention-only error
1	4.2%	12.9%
4	2.0%	5.6%
16	3.9%	4.7%
64	6.9%	6.2%

The attention-only error looks large at batch 1 (~13%) only because attention is tiny there, so a small absolute miss is a big percentage; the *layer-time* error at batch 1 stays ~4%. **Verdict: the model generalizes well.**

(b) Does it capture how attention grows with prompt length? Ratio of attention time at 64K vs 8K tokens, measured against fitted:

TP/batch	measured	fitted
TP2 b1	1.50×	1.46×
TP2 b64	5.94×	5.97×
TP4 b64	4.56×	4.86×

(c) Does it match an independent benchmark? Our predicted B200 time-per-token at batch 4 / 8K tokens is **3.60 ms** (278 tokens/sec/user), versus **~3.58 ms** measured by the external InferenceX serving benchmark — within 0%.

(d) Is the GPU-to-GPU projection sensible? The MI455X has 2.45× the memory bandwidth of the B200, so memory-bound work should speed up by up to 2.45× while fixed overhead doesn’t speed up at all. The predicted speedup therefore grows with batch and context (more memory-bound), as it should:

batch / prompt	B200 time/token (ms)	MI455X time/token (ms)	speedup
1 / 8K	2.197	1.790	1.23×
16 / 8K	2.934	2.095	1.40×
64 / 128K	6.337	3.497	1.81×

(e) **Is the sliding-window attention captured?** Because those layers only look back a fixed 128 tokens, their attention time should be flat regardless of prompt length — and it is (it varies only 1.75× across 8K–256K). The flat fit matches the measured sliding attention to **10.1%** on average.

(f) **Does the overlap adjustment match the profile?** Our predicted full-model time-per-token (36 layers, B200) should match the profile’s own overlap-adjusted numbers at 8K tokens:

config	model TPOT (ms)	profile TPOT (ms)	Δ
TP8 c4	2.350	2.346	+0.2%
TP4 c4	2.700	2.683	+0.6%
TP2 c4	2.932	2.927	+0.2%
TP4 c16	3.455	3.344	+3.3%
TP2 c16	3.967	3.739	+6.1%
TP2 c64	5.994	5.650	+6.1%

The low-batch anchors agree within ~0.5%; the ~6% at the higher-batch settings is ordinary fit scatter, not an overlap error. **Verdict: the overlap model reproduces the measured behavior.**

(g) **How does the model compare to the full serving benchmark?** Same settings as (f), now against the real end-to-end InferenceX numbers (8K tokens), sorted by difference:

config	tp	conc	isl	model TPOT (ms)	InferenceX TPOT (ms)	% diff
TP2 c64 8K	2	64	8192	5.9943	9.3049	-35.6%
TP2 c16 8K	2	16	8192	3.9669	4.6985	-15.6%
TP4 c16 8K	4	16	8192	3.4551	3.6855	-6.3%
TP2 c4 8K	2	4	8192	2.9317	2.9941	-2.1%
TP8 c4 8K	8	4	8192	2.3497	2.3922	-1.8%
TP4 c4 8K	4	4	8192	2.6997	2.5537	+5.7%

The model predicts **lower** times than the real system, more so at high concurrency (~36% at the busiest setting). This is expected, not a bug: the benchmark generates far more tokens per request (1024 vs our 128, so its per-token average carries a longer tail) and includes serving overhead — scheduling, sampling, batching machinery — that our pure layer-compute model leaves out. At low concurrency the two agree within a few percent. **Treat the model as a clean compute estimate, optimistic versus a loaded production server.**

Appendix D. External calibration anchor (Cerebras, all-on-wafer)

Cerebras publishes **~3,000 tokens/sec/user** for GPT-OSS-120B running entirely on the wafer (cerebras.ai). It is a useful real-silicon anchor, but the comparison is **not apples-to-apples** and is easy to misread:

- It is a **single-stream latency** number — one user, batch ≈ 1 — on their production cloud, not a sustained-throughput or cost figure.
- It is **CS-3 running the whole layer on-wafer** (attention *and* the rest, ~ 9 us/layer, no off-wafer hop). Our hybrid keeps attention on the GPU and runs only the rest on **two CS-4 wafers**, paying a **~ 12 us/layer GPU $\leftrightarrow$ CS-4 link** on the critical path; at 131K we reach ~ 863 tok/s/user (~ 32 us/layer, ~ 12 us of it the link).

How to read the $\sim 3.5\times$ gap. On a per-layer basis (~ 9 us vs ~ 32 us) most of it is structural: they pay no off-wafer link and run attention fused on-wafer, while we charge the link and keep attention on the GPU. Even driving the link to zero (co-packaged) leaves a residual gap from fused on-wafer attention plus simply more silicon per stream. So either **we are being conservative** or **they are spending more silicon on one user** — both are consistent with the number.

What it does and doesn't tell us. It is a real-silicon **single-stream latency ceiling at ~ 131 K**. It says **nothing about sustained throughput or \$/token at high concurrency**, where wafer **SRAM capacity** should bind (they are almost certainly SRAM-bound above ~ 131 K) — exactly the regime the hybrid targets. We therefore treat it as a **calibration anchor, not a target**: it bounds our single-stream latency, not our throughput/cost story.

Generated by `measured_report.py`. GPU times from `measured_gpu.py` (measured B200 per-kernel decode times + bottleneck-class projection to MI455X). CS-4 rest + GPU $\leftrightarrow$ CS-4 link from `cs4_offload.py`; hybrid pipeline math from `pipeline_throughput.py`.